On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability

2026. 10.

presenter: **Joohyung Yun**

The Qwen3.8 Model Family and the Scope of This Report

announced Aug 3, 2026 (preview at WAIC in July)

Aug 12	Aug 14	**Aug 26**	Aug 26	Sep 18
Qwen3.8-Max	**Qwen3.8-27B**	**Qwen3.8-Flash-Next**	**Qwen3.8-Flash**	**Qwen3.8-Omni-Flash**
2.4T total · 95B active 92 layers · 512 experts	27B dense · 64 layers vision-language	125B total · 6B active + 51B n-gram tables off-GPU	Flash-Next in production 1M context · built-in tools	text · image · audio · video inherits the Next architecture
open weights	Apache 2.0	open weights	API only	API only
the flagship	the local workhorse	**today · Qwen4 preview**	same model, hosted	omni-modal agent
document that explains how the model is built				
blog post only	blog post only	**this paper · arXiv 2608.30320**	**same paper**	**Omni paper · arXiv 2609.25611**

first introduced in Flash-Next (and headed for Qwen4)

Qwen Sparse Attention ①' · Gated Residual ② · n-gram dictionary ③
Muon recipe, refit hyperparameters, stress-tested stability ④

no series-wide technical report exists (the last one was Qwen3, May 2025)

this paper is the one document that explains how a Qwen3.8 model is built, and it previews the Qwen4 architecture

sources: Hugging Face model cards (Qwen3.8-2.4T-A95B · 27B · Flash-Next), QwenLM/Qwen3.8-Flash-Next README, qwen.ai blog · as of Oct 2026

Design Goal and Headline Result

previous-generation flagship

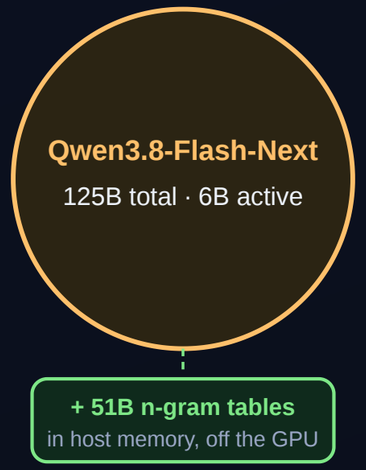


circle area ∝ activated parameters

≈

same quality

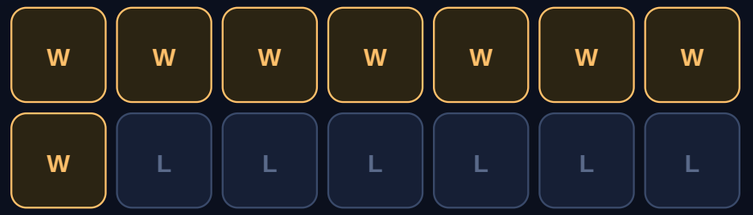
this paper



1/3 the activated parameters

scoreboard vs. Qwen3.7-Plus-Base

14 evaluation benchmarks · base models, before post-training



8 wins

6 losses, each by at most 2.6 points

activated parameters	**1/3**
training tokens	**1/3**
training FLOPs	**≈ 1/9**

Table 11 · base-model comparison

– **goal** · keep the quality of the 397B-A17B flagship at a fraction of its compute budget

Terminology Used in This Talk

context · tokens



… n tokens, the last one is "now"

the text the model holds at once,
counted in tokens (word pieces)

a page ≈ 500 tokens
this model: up to 1M tokens

reader analogy: the pages so far

prefill vs decode

prompt → answer

read all at once — one token at a time

prefill reads the prompt,
decode writes the answer

prefill: compute-bound
decode: memory-traffic-bound

costs are reported per phase

KV cache



one note per past token

notes attention keeps so that
a token can look back exactly

grows with context length
dominates decode memory traffic

① removes it from 3 of 4 layers

MoE · activated params



experts: few run per token

many expert sub-networks,
a router picks a few per token

total params = what is stored
activated = what each token computes

125B total · 6B activated

loss vs benchmark

loss
training score

benchmark
exam score

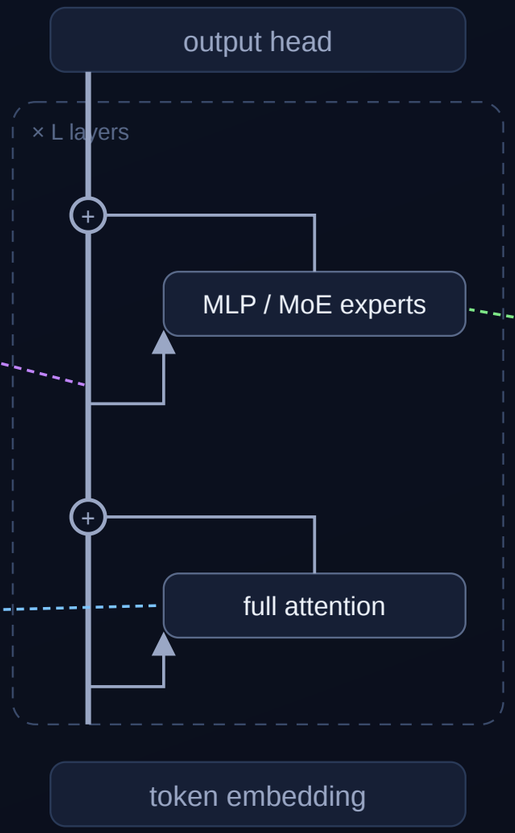
cheap to track vs what matters

loss: how well the next token
is predicted during training

benchmark: accuracy on tasks
the two do not always agree

a recurring theme of this paper

Cost Bottlenecks of the Standard Transformer



2 one narrow stream

all L layers write onto the same line, so what layer 0 wrote has faded by the time layer 15 reads it

3 capacity costs compute

to know more, add parameters; every added parameter in the backbone is more FLOPs on every token

MoE helps (experts on call) but the backbone still pays

1 looking back is quadratic

every token re-reads every earlier token: cost ∝ n^2 and the notes kept per token (KV cache) grow with n

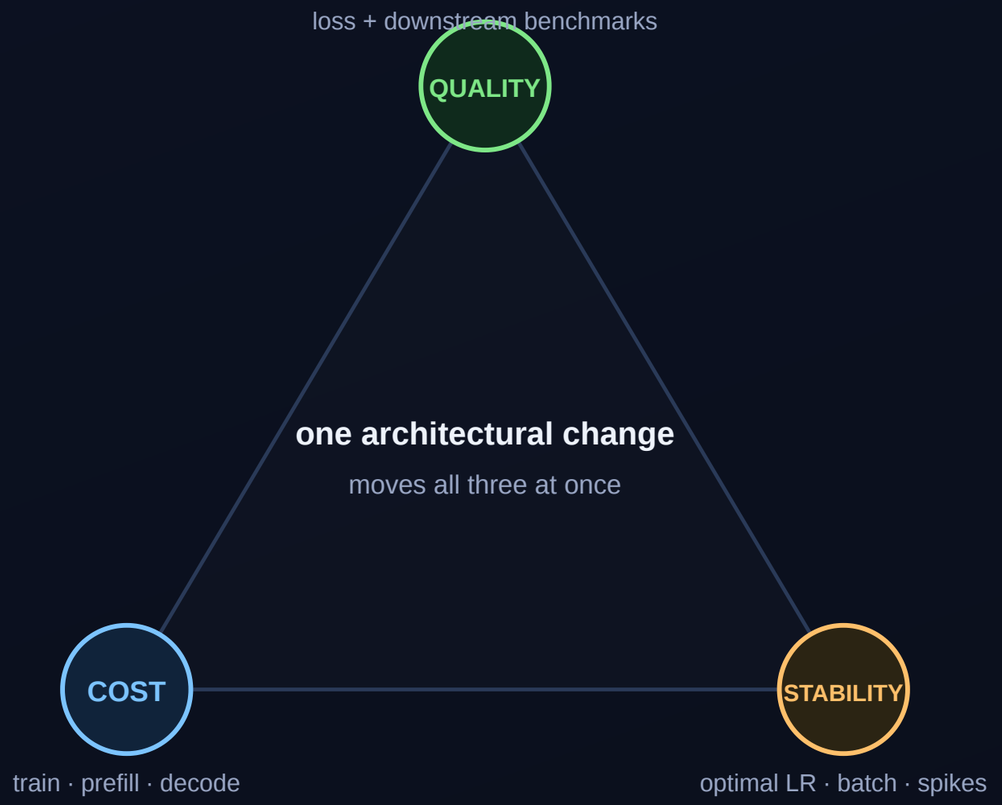
at 1M tokens this is where prefill and decode time goes

4 training breaks at scale

loss spikes → restarts, clipping tricks, and a timid learning rate and batch size (slower convergence)

four bottlenecks → four swaps · and any swap also moves the optimal training recipe (④)

Three Evaluation Axes for Every Architectural Change



rule: a change is kept only if it survives all three

when the scores disagree (four cases in this paper)

n-gram vocabulary ×20 → ×200

loss ↓↓ 1.553→1.526 | exam → flat (MMLU 64.1→64.9) → size by exam

gated residual: make the read data-dependent

loss ≈ (−0.002) | exam ↑ +1.98 points → keep it

read only the 2 strongest lanes (cheaper)

pre-training: fine | after post-training: worse → rejected

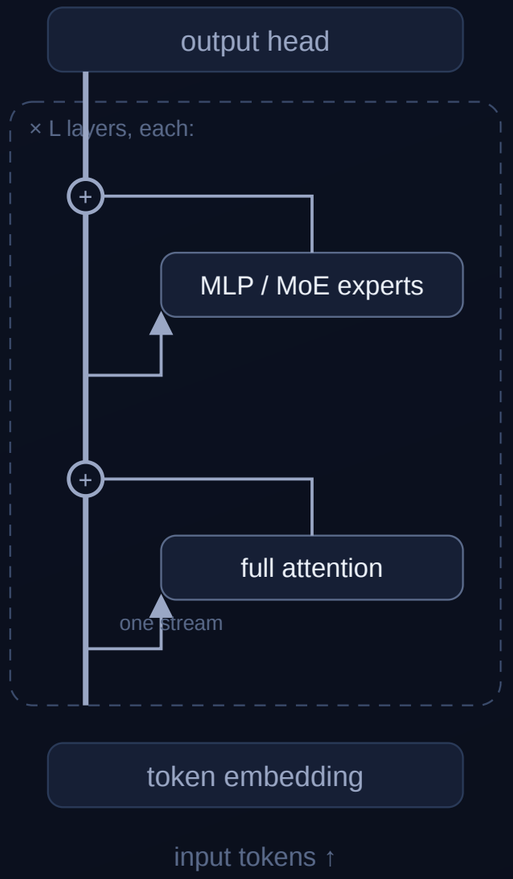
drop positional encoding in attention layers (NoPE)

pre-training: same | generation never stops → keep RoPE

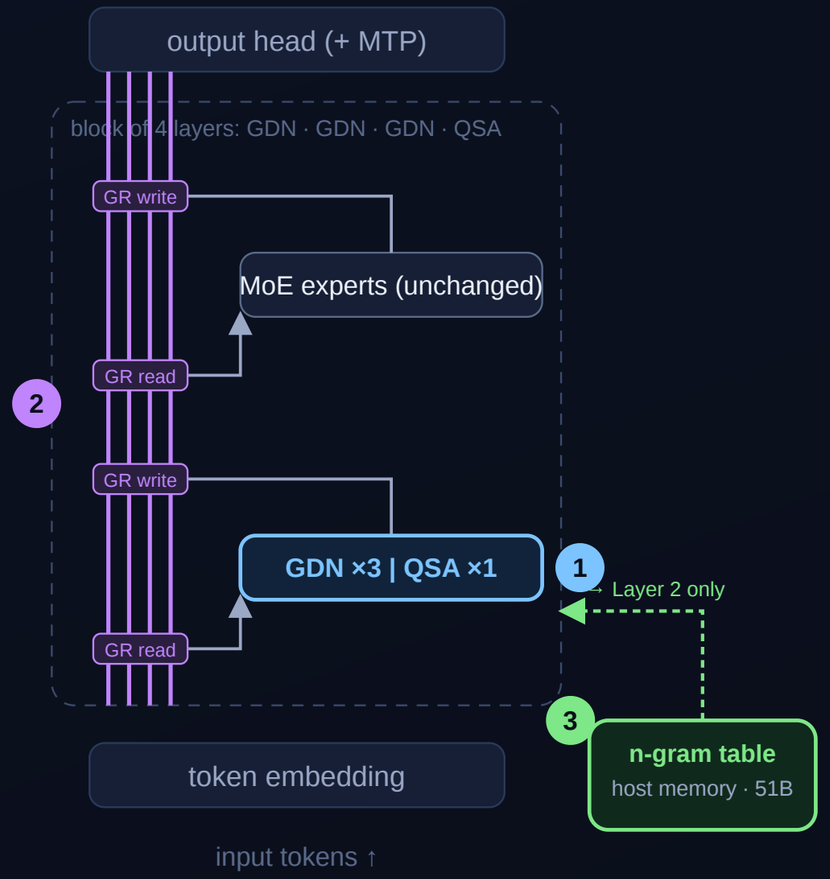
two of the four only show up after post-training: the cheap early signal would have chosen wrong

Overview of the Qwen3.8-Flash-Next Architecture

standard Transformer (Qwen3.5 structure)



Qwen3.8-Flash-Next (same skeleton)



4 training recipe

④a Muon optimizer
matrix-shaped updates on linear maps
AdamW kept for embeddings · router · gates

④b hyperparameters refit
optimum moved: batch ×2 · LR ↑
batch-size warmup dropped

④c stress-tested stability
LR held at 4× optimal → 0 loss spikes
the gate in ② is the key contributor

every swap scored on
quality · cost · stability

① how a token looks back · ② the stream · ③ the dictionary · ④ how it is trained → one part each

PART 2

Architecture

Part 1 Motivation · the goal, the bottlenecks, the three axes

Part 2 Architecture · four swaps

Part 3 Training recipe · optimizer, hyperparameters, stability

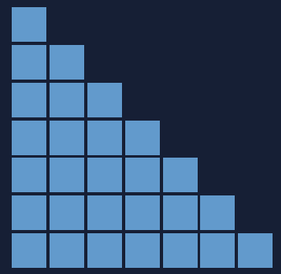
Part 4 Results and takeaways

Full Attention, Sliding Window, and Gated DeltaNet

① token mixer ② residual stream ③ n-gram memory ④ training recipe

full attention

re-read every earlier page

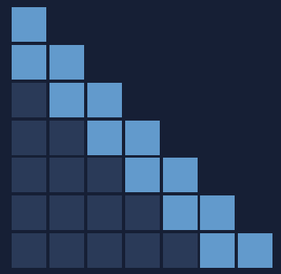


exact recall of any token

cost ∝ n² · notes grow with n

sliding window (SWA)

only the last 128 pages



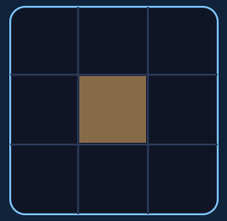
cheap: cost ∝ n · fixed notes

far-away info only trickles through depth

Gated DeltaNet (GDN)

a fixed-size notebook, erase-and-write

erase old,
write new ↗



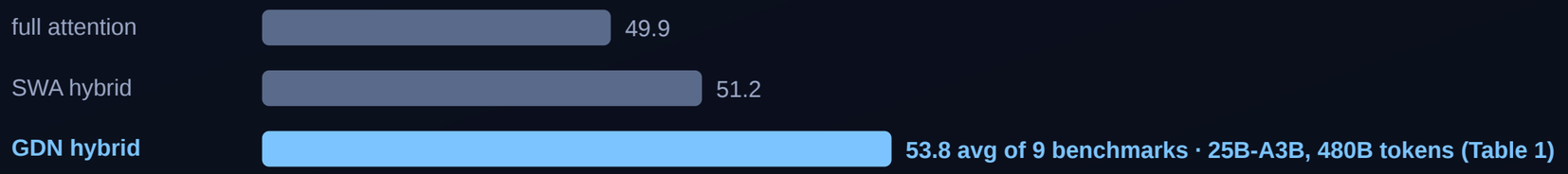
state S:
d × d, fixed

cost ∝ n · fixed memory · content-aware

lossy: cannot recall one exact token

the hybrid: 3 GDN layers for every 1 full-attention layer (exact recall kept, once per four)

GDN GDN GDN ATTN GDN GDN GDN ATTN … attention layers keep RoPE (NoPE: endless generation after post-training)



The Gated Delta Recurrence

a new token arrives with a key k (what it is about) and a value v (what to remember) · four moves on the notebook S

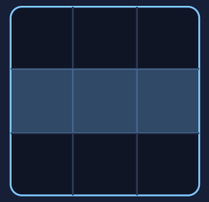
① fade



$S \leftarrow \alpha \cdot S$

α ∈ (0,1): how much to forget, chosen per token by the model

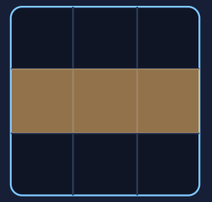
② recall



$\hat{v}$ = what S already holds for k

"have I seen this key before?"

③ write only the error



$S \mathrel{+}= \beta \cdot k\,(v - \hat{v})^{\top}$

repeated keys update, not pile up
β ∈ (0,1): how hard to write

④ read + gate



$y = S^{\top} q \rightarrow \sigma\text{-gate} \rightarrow \text{out}$

sigmoid output gate (bounded): consistently better than SiLU here

$$S_t = \alpha_t \left(I - \beta_t\, k_t\, k_t^{\top} \right) S_{t-1} + \beta_t\, k_t\, v_t^{\top}$$

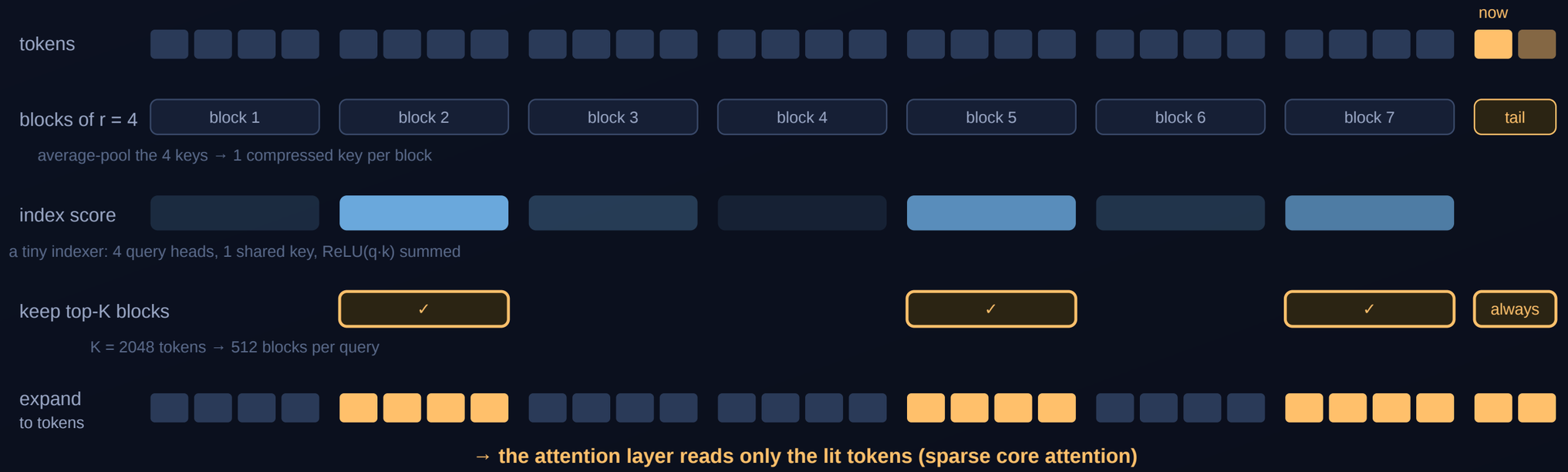
fade · erase what k had · write the new value

- unlike plain linear attention (which only adds), the delta term erases before it writes, so similar keys do not pile up
- q and k are L2-normalized and pass a short causal convolution first (local context before compression)
- kernel: FlashQLA, 2–3× forward and ≈2× backward over the Triton baseline

Qwen Sparse Attention With a Compressed Indexer

① token mixer | ② residual stream | ③ n-gram memory | ④ training recipe

the 1-in-4 full-attention layer still re-reads everything; at 1M tokens it dominates prefill · QSA keeps the layer but reads only what an index points to



→ the attention layer reads only the lit tokens (sparse core attention)

why blocks, not tokens

the index itself was $O(n^2)$ in prior work (DSA); scoring 1 block instead of 4 tokens: $O(n^2) \rightarrow O(n^2/r)$

indexing cost falls with r; r = 4 chosen by ablation

why it fits the hybrid

compression happens inside the layer, so no index sharing across layers is needed (3 GDN layers sit between attention layers and make them dissimilar)

position handling

pool first, then give the block one position (partial RoPE on 64 of 128 dims); the current query keeps its own token position

Training and Evaluation of Qwen Sparse Attention

① token mixer ② residual stream ③ n-gram memory ④ training recipe

introduced at continued pre-training (256K sequences), in two stages

Stage 1 · dense distillation

the indexer learns to imitate full attention
(KL to the teacher, max-pooled to blocks)

backbone frozen · 1,000 steps · ≈2B tokens
warm-up only: indexer alone is not yet usable

→

Stage 2 · sparse training

backbone + indexer trained jointly while
attention reads only the top-K blocks

8,000 steps · ≈200B tokens · KL only over selected blocks
loss gap to full attention ≈ 10^{-4} (Fig. 4)

full attention → QSA, same model



grey = full attention · blue = with QSA · Tables 2–3 · bars scaled 0–100

7 of 8 short-context benchmarks match or improve; the gain grows with context length

kernel-level latency at 1M tokens (vs dense GQA, FlashInfer)

prefill

7.6×

16K-token chunks, batch 1

decode

4.9×

batch 4, with 3 MTP steps

where the speedup comes from

indexer alone: 3.8× prefill · 4.4× decode (r = 4 vs r = 1)
plus sparse core attention over 2,048 of n tokens

speedups start at 64K and grow with sequence length (Fig. 6)

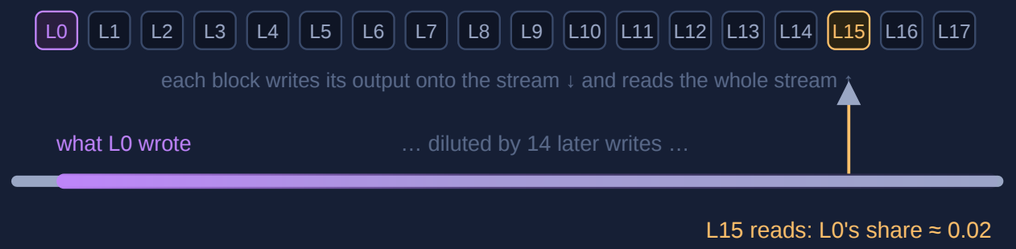
ablation (35B-A3B, RULER up to 1M)

block size 4 matches full attention at 0.25 relative indexer latency
cross-layer index sharing (IndexShare) stays below baseline at 0.5

4 indexer query heads suffice after Stage 2; MTP accepted length 4.06 → 4.07

Limitations of a Single Residual Stream

one stream, pre-norm: every block reads the same line



- norm divides by the whole stream: an early feature competes with all later writes
- one line = one decay rate for every writer; nothing stays fresh for a distant reader
- prior fixes: dense skips, value residuals, cross-layer cache reuse, highway read/write

share at layer 15 of a 20-layer MoE: 0.020 from layer 0 (an equal split ≈ 0.03)

step one: widen the stream to n_r = 4 branches (AltUp-style)

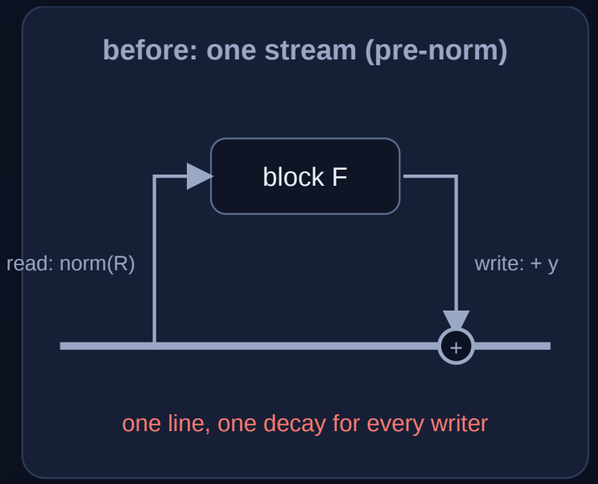


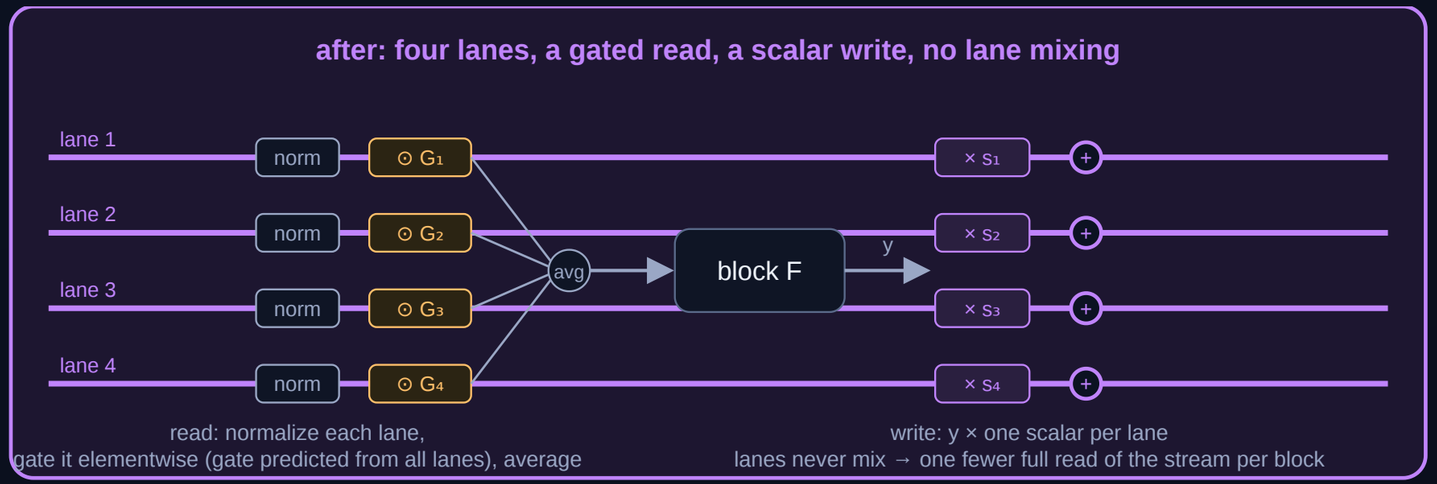
write: round-robin, one branch per block · read: a fixed weighted sum of the four

loss −0.01 on a 25B-A3B MoE (400B tokens), no extra matrix multiply

cost: 4× residual bytes · open question: how much read/write machinery to add on top?

Gated Residual Over a Widened Stream





the ablation ladder (25B-A3B, 560B tokens; avg of 9 benchmarks · final loss, Table 5)

pre-norm, one stream	50.91	1.617
+ four lanes, static read/write	52.49	1.596
+ data-dependent read/write	54.47	1.594
GR: elementwise read, no mixing	**54.66**	**1.590**

⚠ loss ≠ benchmark

static → dynamic: loss −0.002, exam +1.98

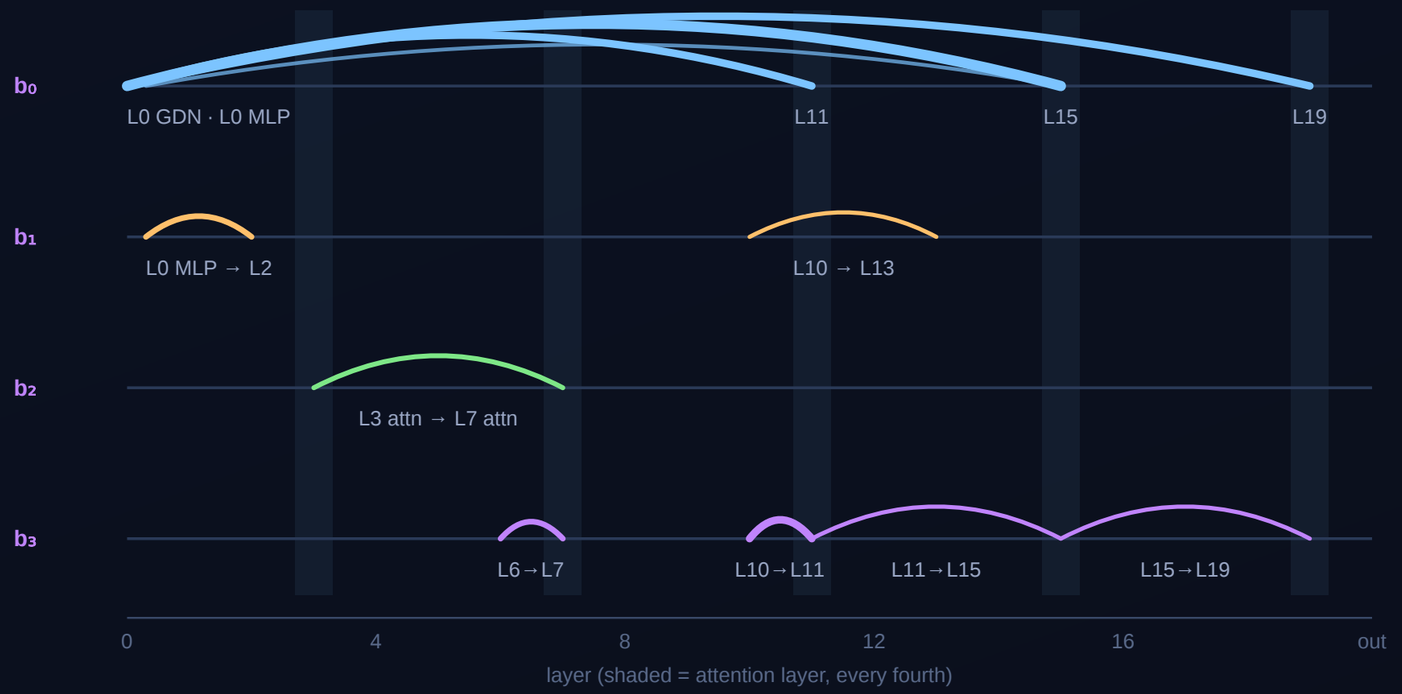
quality ✓ +3.75 · **cost: 4× residual bytes, no mixing read** · **stability ✓ (the gate, Part 3)**

GR = GatedNorm on the widened stream; replaces the pre-norm

Cross-Layer Paths Carried by the Residual Branches

① token mixer · ② residual stream · ③ n-gram memory · ④ training recipe

which earlier block feeds which later block, GR minus a single-stream reference (20-layer MoE, Fig. 7) · arc width = extra share, only Δ ≥ 0.05 drawn (21 of 780 pairs)



b₀ carries the long-range paths: written at layer 0, barely touched after, read by every attention layer past 10

inference: FP8 residual state

gates bound what is written, so lane values stay in a narrow range → lanes stored in FP8: half the bytes of BF16, almost no quality loss

read and write each fused into one kernel; group RMSNorm folded into the read → the stream is traversed once per block

⚠ rejected: read only the two strongest lanes

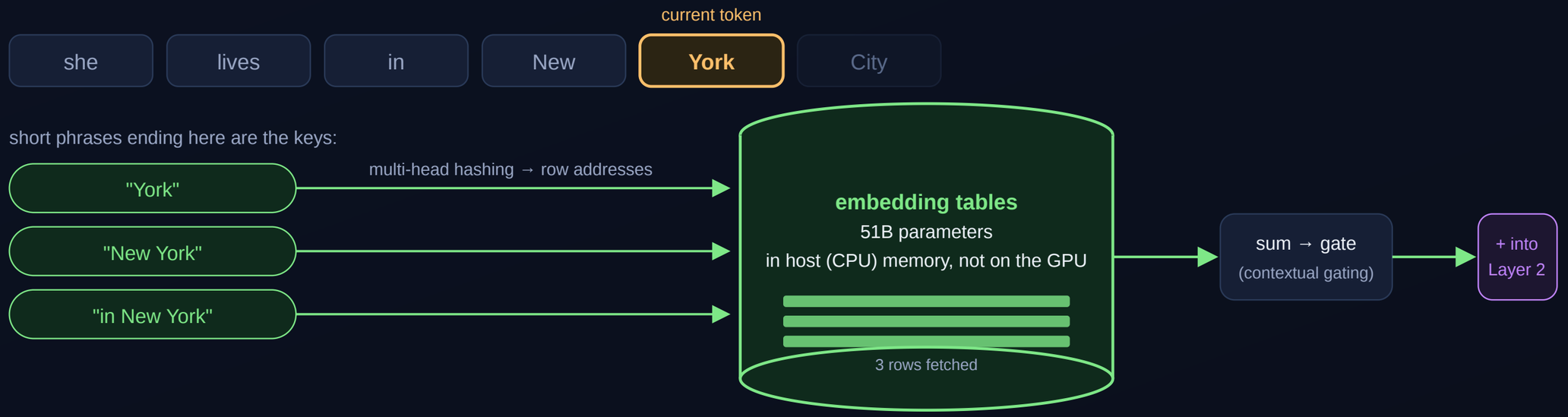
trained models write mostly into two lanes, so a sparse read looked free: pre-training loss and benchmarks unchanged

quality degraded clearly after post-training

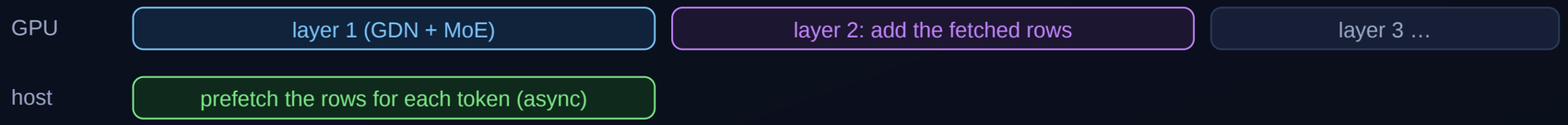
pre-training metrics alone would have chosen wrong; varying sparsity per layer did not fix it

- **one branch** carries long-range paths (typical skip 10.9 layers) · the other three stay local (3.4–3.9)
- layer-0 GDN → layer-15 attention: share **0.020** → **0.138** · attention layers are where the long paths land

N-gram Embedding Layer With Host-Memory Tables



why it is almost free: addresses depend on the tokens alone, so the fetch runs while layer 1 computes



FLOPs per token ≈ 0
a lookup, not a matrix multiply

capacity grows off the accelerator
125B on the GPU + 51B on the shelf

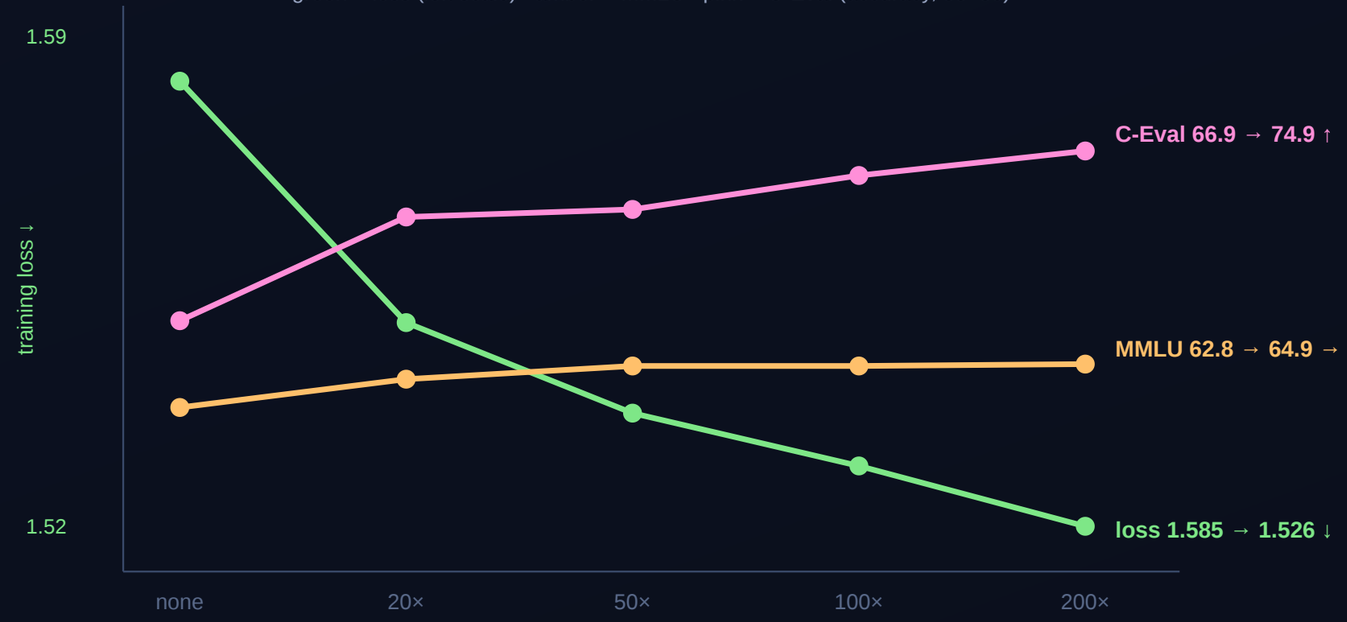
one layer is enough (ablation, next)
placement barely matters; Layer 2 hides the fetch

Ablations of the N-gram Embedding Layer

① token mixer · ② residual stream · ③ n-gram memory · ④ training recipe

vocabulary scaled 20× → 200× the base tokenizer, MoE budget fixed (Table 9)

green = loss (left scale) · amber = MMLU · pink = C-Eval (accuracy, 55–80)



MATH 32.5 → 37.4 then 35.3 · GSM8K 59.2 → 65.1 then 63.0 · MMMLU 54.1 → 56.6 then 55.8

gains over the baseline are broad, but most benchmarks saturate or fluctuate as the vocabulary grows

placement (Table 7, same parameter budget)

a single layer at depth 1, 2, 3, 4, 10, 15 or 25: avg 46.6–47.9
vs 45.4 without · splitting across two layers adds nothing

→ one layer, at Layer 2 so the fetch overlaps layer 1

experts vs n-gram slots (Table 8, fixed total parameters)

trading MoE experts for n-gram slots: loss dips at 10× (1.202 → 1.197)
but no benchmark gain over the MoE-only baseline

→ distinct roles; keep the experts, add the tables on top

⚠ loss ≠ benchmark

loss falls monotonically with vocabulary size; exam scores saturate,
except Chinese (C-Eval, CMMLU), which keep improving

size the vocabulary by downstream accuracy, not by loss;
compression tricks (normalization, non-uniform orders) gave no consistent gain

Summary of the Four Architectural Components

component	bottleneck	swap	price	gain	axes
① GDN hybrid	look-back cost ∝ n², KV cache grows with n	3 of 4 layers use a fixed-size erase-and-write memory; attention kept once per four, with RoPE	GDN layers cannot recall one exact token	avg 49.9 → 53.8 on 9 benchmarks · FlashQLA 2–3× fwd	Q ✓ C ✓ S ✓
①' QSA	the remaining attention layers dominate prefill at 1M	4-token block index (4 query heads) + top-512 blocks → sparse core attention	two-stage continued pre-training (≈200B tokens)	75.9 → 76.8 · RULER 512K–1M +2.9 · 7.6× prefill, 4.9× decode at 1M	Q ✓ C ✓ S ✓
② Gated Residual	one stream: early writes fade before late readers	4 lanes · elementwise gated read from all lanes · scalar write · no lane mixing	4× residual bytes (FP8 halves it); sparse read rejected after post-training	avg 50.9 → 54.7 · zero loss spikes at 4× LR	Q ✓ C ✓ S ✓✓ ⚠
③ n-gram memory	more capacity = more FLOPs per token	51B hashed phrase tables in host memory, prefetched, added at Layer 2	host-memory bandwidth; loss and exams disagree when scaling	avg 45.4 → 47.9 at ≈0 extra FLOPs · Chinese keeps rising with size	Q ✓ C ✓ ⚠

PART 3

Training Recipe

Part 1 Motivation · the goal, the bottlenecks, the three axes

Part 2 Architecture · four swaps

Part 3 Training recipe · optimizer, hyperparameters, stability

Part 4 Results and takeaways

Muon Optimizer and Parameter Assignment

① token mixer ② residual stream ③ n-gram memory **④ training recipe**

how a weight matrix W (A × B) is updated from its momentum M



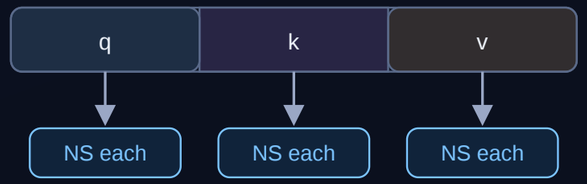
momentum (Nesterov 0.95)

AdamW · elementwise
each entry scaled by its own running RMS; no notion of the matrix as a map

Muon · the whole matrix
orthogonalize M (Newton–Schulz, 8 steps, Polar Express coefficients):
all singular values → 1, then scale by 0.2·√max(A, B)

every direction moves equally · update RMS independent of matrix shape

fused matrices are split before orthogonalization



one stored matrix, but three (per-head) operators

then gathered back (per head; gate/up for fc1)

orthogonalizing the concatenation mixes singular directions across blocks, with the wrong shape for the scale

splitting improves loss and benchmarks; per-head scalar projections (decay, β) stay on AdamW

Muon: weights that act as linear maps

- attention q / k / v / output projections
- GDN input and output projections
- expert fc1 / fc2, routed and shared
- n-gram key / value projections

Canzona balances NS FLOPs across ranks
one CUDA graph per optimizer step

AdamW: the rest

- input embeddings, output head
- MoE router (Muon destabilizes it early)
- GR low-rank projections (elongated)
- output gates of attention and GDN

n-gram tables: Adam, no weight decay
router rows = independent expert scores

why 8 Newton–Schulz steps

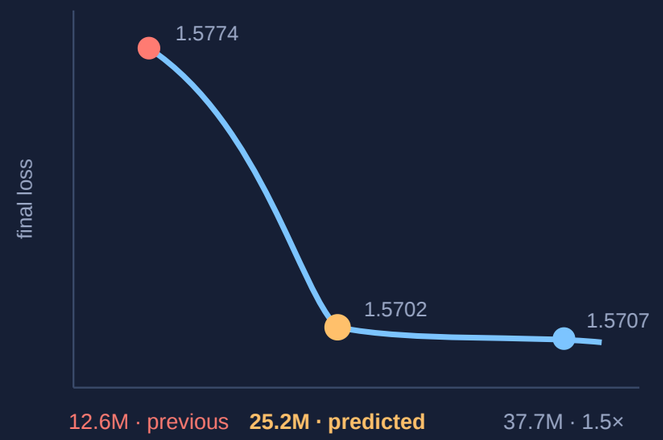
more accurate orthogonalization than fewer steps; in the stress test it reduces both the magnitude and the frequency of gradient-norm spikes

NS cost ≈ 4K · max(A,B) · min(A,B)² FLOPs: cubic in the short side, so an equal-element partition across ranks leaves stragglers (hence Canzona); after splitting, a layer has ~100 small sub-matrices, so the step is launch-bound (hence the CUDA graph)

Hyperparameter Scaling Under the New Recipe

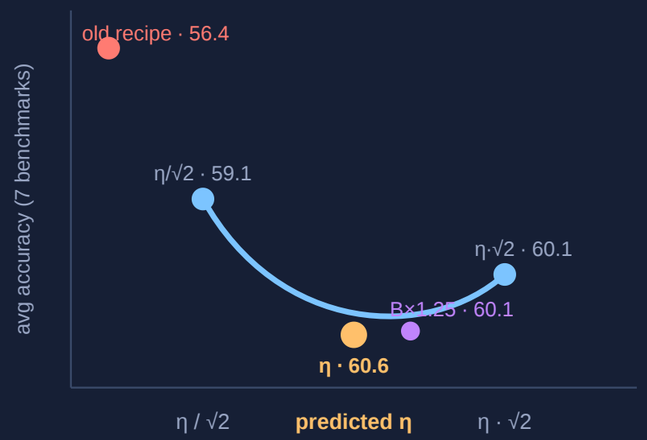
① token mixer · ② residual stream · ③ n-gram memory · **④ training recipe**

batch size · 10.8B-A0.89B, 4T tokens



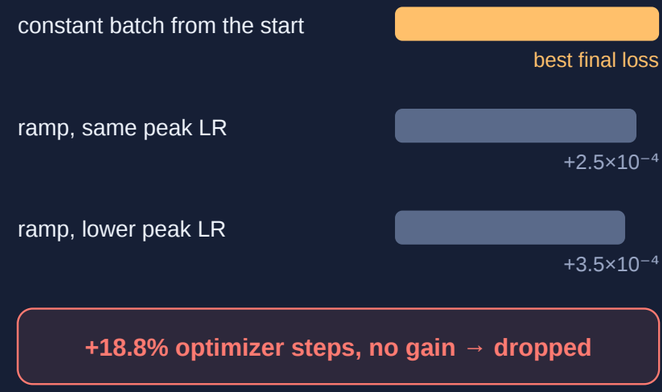
steep below the prediction, flat above: −7.2×10⁻³ vs the old recipe

learning rate · 156B-A7B (48 layers), 419B tokens



a flat bowl over √2 either way · old recipe ends 7.8×10⁻³ higher in loss

batch-size warmup · 6.3M → 25.2M ramp



+18.8% optimizer steps, no gain → dropped

small early batches add gradient noise; the extra steps' transient advantage disappears as the learning rate decays · no stability change

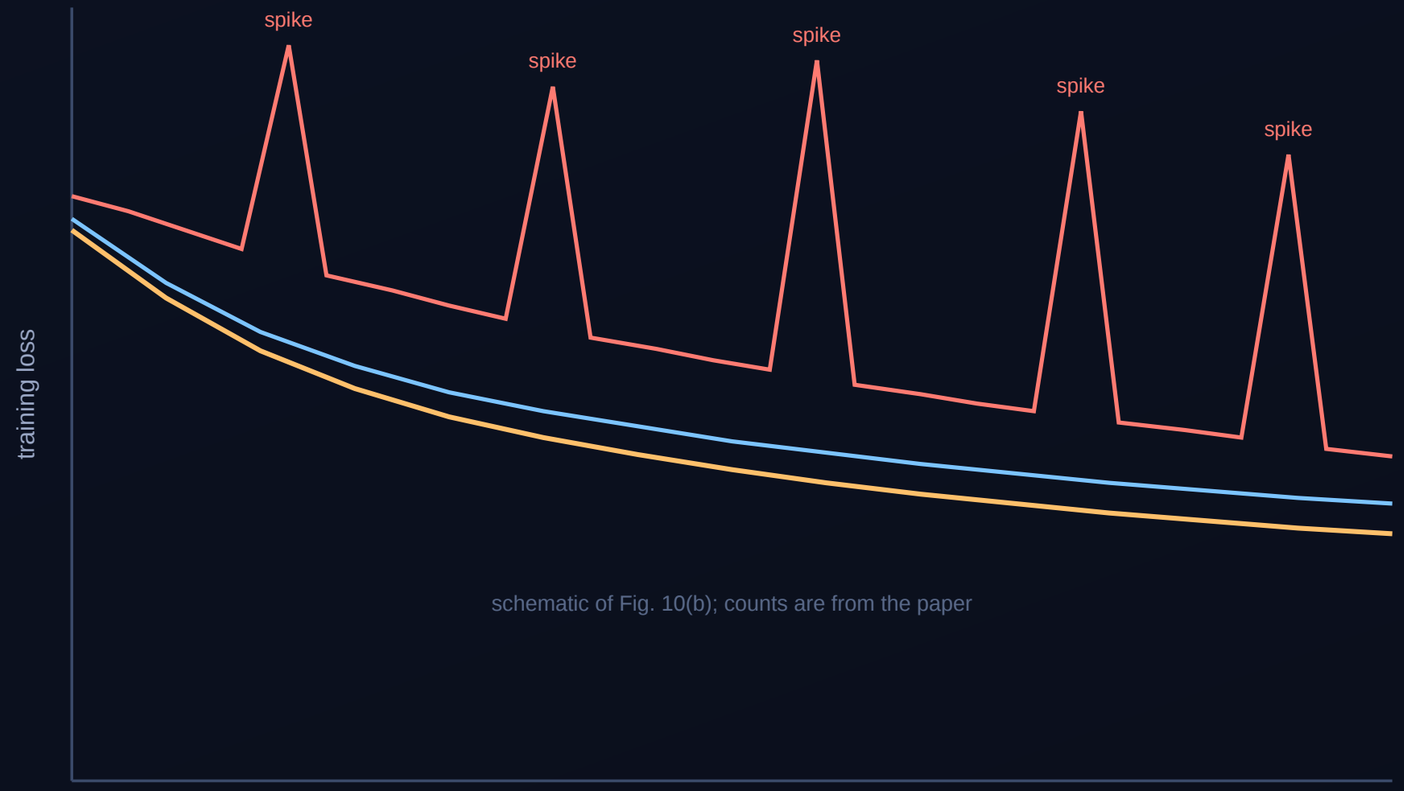
verification where each prediction is most sensitive: batch size on a small model trained long (too-large batch hurts), learning rate on a large model trained short (instability risk)

gradient clipping never engages after warmup; at the predicted optimum the max pre-clip gradient norm is 28% of the threshold (51% under the old recipe)

– **new scaling-law fit** · larger batch and learning rate, slower LR decay with model size · Muon keeps data efficiency at large batch

Stability Stress Test at Elevated Learning Rates

instabilities that appear at trillion-parameter scale are reproduced in a 28-layer 25B-A3B model by holding the learning rate at a multiple of its optimum, with no decay



schematic of Fig. 10(b); counts are from the paper

training steps (0 → 20k) · LR = 8×10^{-3} held constant = 4× optimal

AdamW · Qwen3.5 structure
183 loss spikes per 10k steps
clipper engaged on 213 of 19,932 steps

Muon · Qwen3.5 structure
0.2 spikes per 10k at 2×; calm at 4×
never crosses the clip threshold

Muon + Gated Residual
0 loss spikes, even at 4×
same batch, same data, same clip rule

criterion: at least as stable as the old recipe under equal stress · passed with margin

spike = a step exceeding the 201-step rolling median by 0.1 · clip threshold 0.5 · at 2× the AdamW baseline already spikes 4.3 per 10k steps

Role of the Gate in Training Stability

single-variable pair · 3× optimal LR, AdamW

same structure, data order and optimizer; only GatedNorm toggled

loss spikes / 10k steps

gate off · 32.0

gate on · 3.2

clip-threshold crossings

gate off · 256

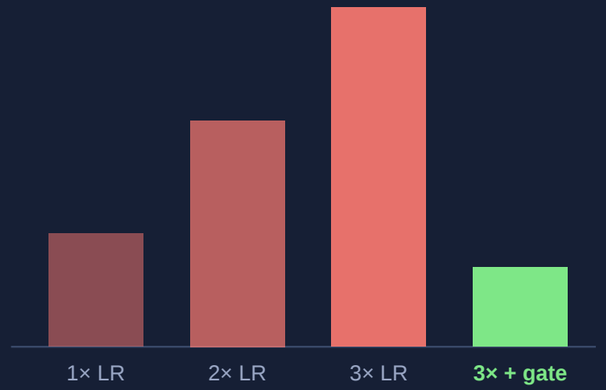
gate on · 20

$$\text{GatedNorm}(u) = \text{RMSNorm}(u) \odot \sigma(W_2\, \text{SiLU}(W_1\, \text{RMSNorm}(u)))$$

the same gate GR carries inside its read (Fig. 12)

activation outliers vs learning rate

max transformer output at layer 20, ungated baseline ladder



outliers grow ≈ proportionally with LR; spikes grow much faster

with the gate at 3×, outliers fall below the ungated 1× level

production run · first 276B tokens, shipped LR

Qwen3.5+Muon · +GR · full recipe (8× the stress-test scale)

loss at 276B tokens	**−0.026 (GR) · −0.058 (full)**
grad norm median / p99.9	**0.097/0.298 → 0.043/0.066**
gradient-norm std in a 1000-step window	**4.3–4.7× lower**
clip-threshold crossings	**Muon alone only**
residual activation maximum	**markedly lower at every depth**

full-scale run: no loss spike, no gradient anomaly

without qk-clip or SwiGLU-clip

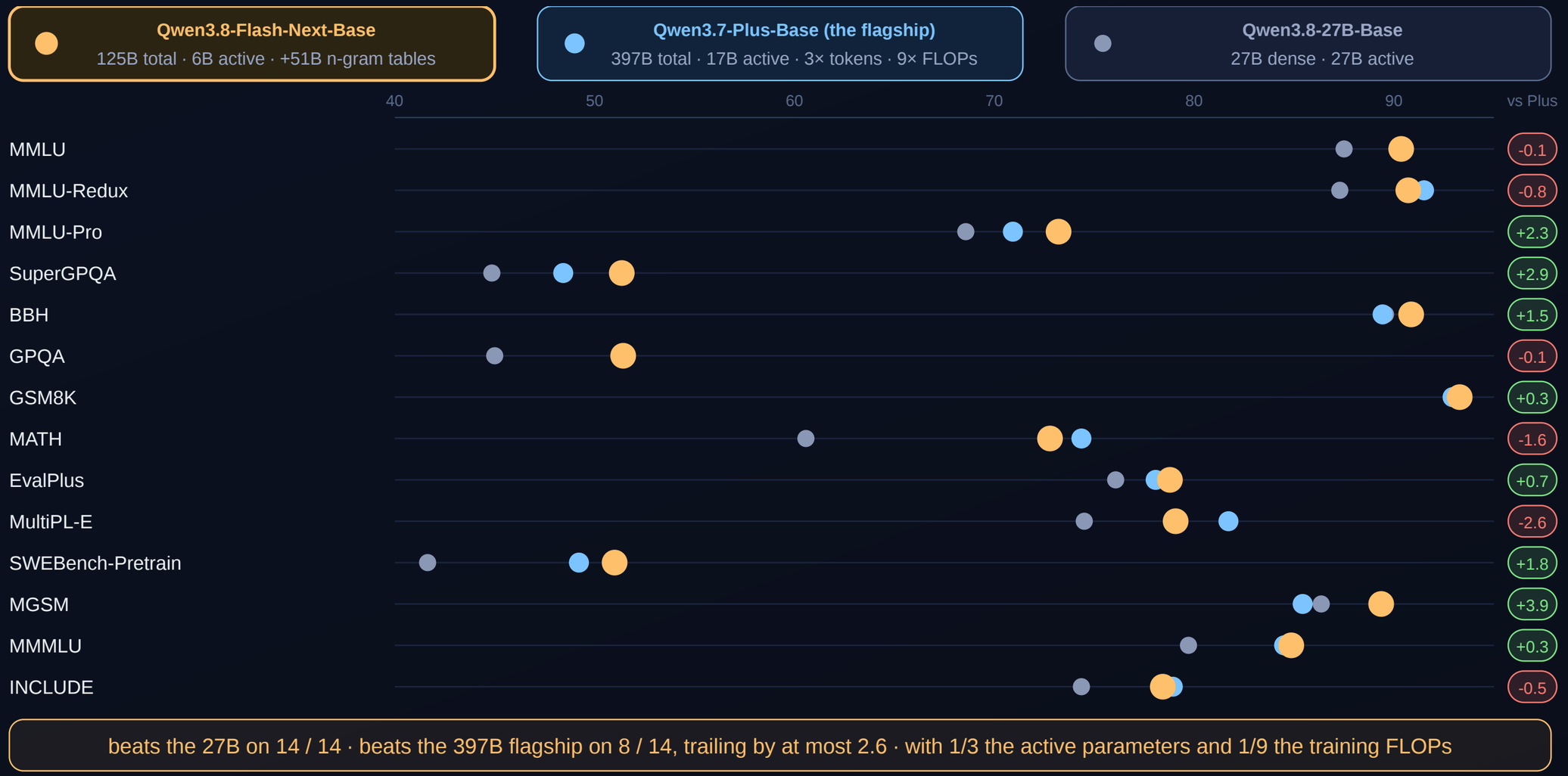
final norm before the LM head fused into a gated read: lower still

at 2× LR the Muon runs show a higher median gradient norm and larger activations than AdamW, yet far fewer spikes; adding GR cuts spike frequency, magnitude and outliers

eight Newton–Schulz steps rather than fewer were chosen for the same reason: fewer and smaller gradient-norm spikes under stress

– training at high learning rates needs a **rescaling mechanism** · without a gate the network grows activation outliers (fragile); the multiplicative gate supplies it directly

Base Model Evaluation on 14 Benchmarks



Efficiency Accounting Across Training, Prefill, and Decode

training budget vs the 397B-A17B flagship: **1/3** activated parameters · **1/3** tokens · **≈ 1/9** FLOPs · inference: fewer activated parameters, and the per-component savings below

component	training	prefill (reading the prompt)	decode (writing the answer)
① GDN hybrid	FlashQLA kernel: 2–3× forward, ≈2× backward over the Triton baseline	3 of 4 layers linear in n instead of n^2	fixed-size recurrent state: no KV cache growth in 3 of 4 layers
①' QSA	fused kernel computes sparse attention and the indexer KL loss without intermediates	indexer $O(n^2) \rightarrow O(n^2/r)$ · attention module **7.6×** faster at 1M (chunked, batch 1)	**4.9×** at 1M (batch 4, 3 MTP steps) · MTP reuses the top-k indices across speculative steps
② Gated Residual	stability margin → larger learning rate and batch, no warmup, no clipping tricks	cost is memory traffic of the widened state: no lane-mixing read per block · lanes stored in **FP8** (½ the bytes) · read and write fused into one kernel each	
③ n-gram memory	tables on Adam without weight decay; 51B parameters never touch the accelerator	≈ 0 FLOPs per token · rows prefetched from host memory asynchronously, hidden behind layer 1	
④ Muon	Newton–Schulz cost balanced across data-parallel ranks (Canzona) · optimizer step captured in one CUDA graph	no inference cost	

Takeaways

Four bottlenecks, four components

Quadratic look-back → GDN hybrid and QSA. One narrow stream → Gated Residual. Capacity priced in FLOPs → n-gram tables on the shelf. Each swap addresses one cost and keeps the skeleton.

The protocol is the contribution

Every change scored on quality, cost and stability, at a scale where the full evaluation is affordable. Loss alone would have taken four wrong turns: sparse lane reads, NoPE, an oversized n-gram vocabulary, batch-size warmup.

Open problem: evaluation throughput

Two of the four disagreements surfaced only after post-training. A cheaper mid-scale probe that predicts post-training order would make the design space far more searchable. What should such a probe measure?

One line · flagship quality at a ninth of the compute, obtained not by one trick but by treating **architecture, efficiency and optimization as one coupled system**, and by checking every shortcut on all three axes